

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №8

Тема: Алгоритми сортування. Загальна класифікація та принципи роботи

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Ознайомитися із загальною класифікацією алгоритмів сортування за різними критеріями (складність, стійкість, використання додаткової пам'яті, адаптивність), опанувати принципи роботи сортування вибором та сортування вставками, дослідити їхню ефективність та властивість стійкості на практиці.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Постановка задачі сортування. Дано послідовність із n елементів. Потрібно переставити елементи так, щоб вони утворили неспадну (або незростаючу) послідовність за деяким ключем порівняння.

2. Класифікація алгоритмів сортування:

- **За способом порівняння:**
 - **Порівняльні алгоритми** (comparison-based) — впорядковують елементи, порівнюючи пари між собою (бульбашкове, вибором, вставками, злиттям, швидке, пірамідальне сортування). Для них доведено теоретичну нижню межу складності $\Omega(n \log n)$ у найгіршому випадку (доведення методом дерева рішень: будь-який алгоритм, що впорядковує n елементів лише порівняннями, повинен виконати не менше $\log_2(n!) \approx n \log_2 n$ порівнянь).
 - **Непорівняльні алгоритми** — використовують додаткову інформацію про структуру ключів (лічильникове сортування, порозрядне сортування, сортування кошиками), можуть досягати складності $O(n)$ за певних обмежень на діапазон значень ключів.
- **За місцем розташування даних:**
 - **Внутрішнє сортування** — усі дані вміщуються в оперативній пам'яті.
 - **Зовнішнє сортування** — дані не вміщуються в пам'яті одночасно, обробляються частинами із зовнішньої пам'яті (наприклад, зовнішнє сортування злиттям для великих файлів).
- **За використанням додаткової пам'яті:**
 - **"На місці" (in-place)** — потребує лише $O(1)$ (або $O(\log n)$) додаткової пам'яті (бульбашкове, вибором, вставками, швидке сортування).

- **Не "на місці"** — потребує суттєвого обсягу додаткової пам'яті, як правило $O(n)$ (сортування злиттям у класичній реалізації).
- **За стійкістю (stability):**
 - **Стійкий алгоритм** — зберігає відносний порядок рівних за ключем елементів (важливо, якщо потрібно сортувати за кількома критеріями послідовно). Приклади: сортування вставками, сортування злиттям, бульбашкове сортування.
 - **Нестійкий алгоритм** — не гарантує збереження відносного порядку рівних елементів. Приклад: сортування вибором, швидке сортування (у стандартній реалізації), пірамідальне сортування.
- **За адаптивністю:** адаптивний алгоритм працює швидше на частково відсортованих даних (сортування вставками — яскравий приклад: $O(n)$ на вже відсортованому масиві), неадаптивний — виконує приблизно однакову кількість операцій незалежно від початкового стану даних (сортування вибором завжди виконує $\Theta(n^2)$ порівнянь незалежно від початкового порядку).

3. Порівняльна таблиця основних алгоритмів:

Алгоритм	Складність (сер.)	Складність (найгірший)	Стійкий?	In-place?	Адаптивний?
Бульбашкове сортування	$O(n^2)$	$O(n^2)$	Так	Так	Так
Сортування вибором	$O(n^2)$	$O(n^2)$	Ні	Так	Ні
Сортування вставками	$O(n^2)$	$O(n^2)$	Так	Так	Так
Сортування Шелла	$O(n \log^2 n)$ (залежить від кроків)	$O(n^2)$	Ні	Так	Частково
Швидке сортування (QuickSort)	$O(n \log n)$	$O(n^2)$	Ні	Так	Ні
Сортування злиттям (MergeSort)	$O(n \log n)$	$O(n \log n)$	Так	Ні ($O(n)$ пам'яті)	Ні
Пірамідальне сортування (HeapSort)	$O(n \log n)$	$O(n \log n)$	Ні	Так	Ні
Лічильникове сортування	$O(n + k)$	$O(n + k)$	Так	Ні	—

(k — діапазон значень ключів для лічильникового сортування)

Хід роботи

Завдання 1. Класифікація алгоритмів сортування

Заповнити таблицю для восьми алгоритмів сортування (бульбашкове, вибором, вставками, Шелла, швидке, злиттям, пірамідальне, лічильникове), вказавши: асимптотичну складність у середньому й найгіршому випадках; чи є алгоритм стійким; чи виконується "на місці" (in-place); чи є адаптивним.

Порядок виконання:

1. Створіть таблицю зі стовпцями: "№", "Алгоритм", "Складність (сер.)", "Складність (найгірший)", "Стійкий?", "In-place?", "Адаптивний?".
2. Заповніть таблицю на основі теоретичних відомостей роботи.
3. Для кожного алгоритму, який ще детально не вивчався (Шелла, швидке, злиття, піраміда, лічильникове), коротко (одним реченням) занотуйте принцип його роботи — це знадобиться в наступних практичних роботах, де ці алгоритми розглядатимуться докладно.
4. Позначте зірочкою ті алгоритми, які будуть детально вивчені в наступних практичних роботах курсу (бульбашкове — ПР№13, Шелла — ПР№14, швидке — ПР№15).

Завдання 2. Дослідження стійкості сортування вручну

Дано масив записів (ключ сортування, мітка): (3,А), (1,Б), (3,В), (2,Г), (1,Д). Потрібно вручну відсортувати масив за зростанням ключа методом вибору та простежити, чи зберігається відносний порядок записів з однаковим ключем (наприклад, чи (3,А) залишиться перед (3,В), а (1,Б) — перед (1,Д)).

Порядок виконання:

1. Випишіть вихідний масив у вигляді послідовності пар (ключ, мітка): (3,А), (1,Б), (3,В), (2,Г), (1,Д).
2. Виконайте перший прохід сортування вибором: знайдіть елемент із мінімальним ключем серед усіх п'яти (це один із елементів з ключем 1 — (1,Б) або (1,Д), залежно від того, який зустрічається першим при лінійному переборі) і поміняйте його місцем з першим елементом масиву. Зафіксуйте результат обміну.
3. Продовжте виконання алгоритму вибором для решти елементів (позиції 2–5), щоразу знаходячи мінімум серед невідсортованої частини й обмінюючи його з поточною позицією; записуйте стан масиву після кожного проходу.
4. Після завершення сортування порівняйте відносний порядок елементів з однаковим ключем 1 (Б і Д) та однаковим ключем 3 (А і В) у відсортованому масиві з їхнім порядком у вихідному масиві.
5. Зробіть висновок: чи змінився відносний порядок хоча б однієї пари однакових ключів через прямий обмін місцями елементів (характерна причина нестійкості сортування вибором).
6. Повторіть міркування (без повного трасування) для сортування вставками: поясніть текстово, чому цей метод, який вставляє елемент, зсуваючи більші елементи праворуч (а не обмінюючи місцями через далеку відстань), зберігає відносний порядок однакових ключів, тобто є стійким.

Завдання 3. Програмна реалізація сортування вибором

Реалізувати мовою Python алгоритм сортування вибором з підрахунком кількості порівнянь і обмінів, перевірити коректність порівнянням з вбудованою функцією `sorted()`.

Порядок виконання:

1. Напишіть функцію `selection_sort(arr)`, яка створює копію масиву (щоб не змінювати оригінал) і заводить лічильники `comparisons = 0`, `swaps = 0`.
2. У зовнішньому циклі `for i in range(n-1):` заведіть змінну `min_idx = i` (індекс поточного мінімуму).
3. У внутрішньому циклі `for j in range(i+1, n):` порівнюйте `a[j]` з `a[min_idx]`, збільшуючи `comparisons` на кожному порівнянні; якщо `a[j] < a[min_idx]`, оновіть `min_idx = j`.

- Після завершення внутрішнього циклу, якщо `min_idx != i`, поміняйте місцями `a[i]` та `a[min_idx]` і збільшіть `swaps`.
- Поверніть відсортований масив та лічильники `comparisons`, `swaps`.
- Перевірте функцію на невеликому тестовому масиві з повторюваними значеннями, порівнявши результат із вбудованою `sorted(arr)`.

Завдання 4. Програмна реалізація сортування вставками та порівняльний аналіз

Реалізувати сортування вставками з підрахунком порівнянь, порівняти кількість порівнянь обох алгоритмів (вибором і вставками) на випадкових масивах різного розміру, а також дослідити ефект адаптивності сортування вставками на майже відсортованих даних.

Порядок виконання:

- Напишіть функцію `insertion_sort(arr)`, яка створює копію масиву та лічильники `comparisons = 0`, `swaps = 0`.
- У циклі `for i in range(1, n)`: збережіть поточний елемент `key = a[i]` і заведіть `j = i - 1`.
- У вкладеному циклі `while j >= 0`: порівнюйте `a[j]` з `key` (збільшуючи `comparisons`); якщо `a[j] > key` — зсуньте `a[j+1] = a[j]`, зменшіть `j` на 1 і збільшіть `swaps`; інакше перервіть цикл.
- Після завершення внутрішнього циклу вставте збережений `key` на позицію `a[j+1]`.
- Поверніть відсортований масив і лічильники.
- Для кількох розмірів `n` (наприклад, 100, 300, 600, 1000, 1500) згенеруйте випадковий масив і порахуйте кількість порівнянь для `selection_sort` та `insertion_sort`.
- Для тих самих розмірів створіть **майже відсортований** масив (візьміть відсортований масив і зробіть у ньому лише 5 випадкових перестановок пари елементів) та порахуйте кількість порівнянь `insertion_sort` на цих даних.
- Побудуйте графік залежності кількості порівнянь від `n` для трьох серій: вибір (випадковий масив), вставки (випадковий масив), вставки (майже відсортований масив).
- Проаналізуйте графік і поясніть, чому крива "вставки (майже відсортований)" залишається значно нижчою за інші — це і є прояв адаптивності алгоритму.

Приклад реалізації мовою Python:

```
python
import random

def selection_sort(arr):
    a = arr.copy()
    n = len(a)
    comparisons = swaps = 0
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            comparisons += 1
            if a[j] < a[min_idx]:
                min_idx = j
        if min_idx != i:
            a[i], a[min_idx] = a[min_idx], a[i]
            swaps += 1
    return a, comparisons, swaps

def insertion_sort(arr):
    a = arr.copy()
    n = len(a)
```

```

comparisons = swaps = 0
for i in range(1, n):
    key = a[i]
    j = i - 1
    while j >= 0:
        comparisons += 1
        if a[j] > key:
            a[j + 1] = a[j]
            j -= 1
            swaps += 1
        else:
            break
    a[j + 1] = key
return a, comparisons, swaps

# Перевірка коректності
test = [5, 2, 9, 1, 5, 6, 3]
print("Сортування вибором: ", selection_sort(test)[0])
print("Сортування вставками:", insertion_sort(test)[0])
print("sorted() Python:   ", sorted(test))

# Порівняння на випадкових і майже відсортованих масивах
for n in [100, 300, 600, 1000, 1500]:
    arr_random = [random.randint(0, 10000) for _ in range(n)]
    _, c_sel, _ = selection_sort(arr_random)
    _, c_ins, _ = insertion_sort(arr_random)

    arr_nearly = sorted(arr_random)
    for _ in range(5):
        i, j = random.randint(0, n-1), random.randint(0, n-1)
        arr_nearly[i], arr_nearly[j] = arr_nearly[j], arr_nearly[i]
    _, c_ins_nearly, _ = insertion_sort(arr_nearly)

    print(f"n={n:5d}: вибір(вип.)={c_sel:7d} вставки(вип.)={c_ins:7d} вставки(майже відсорт.)={c_ins_nearly:5d}")

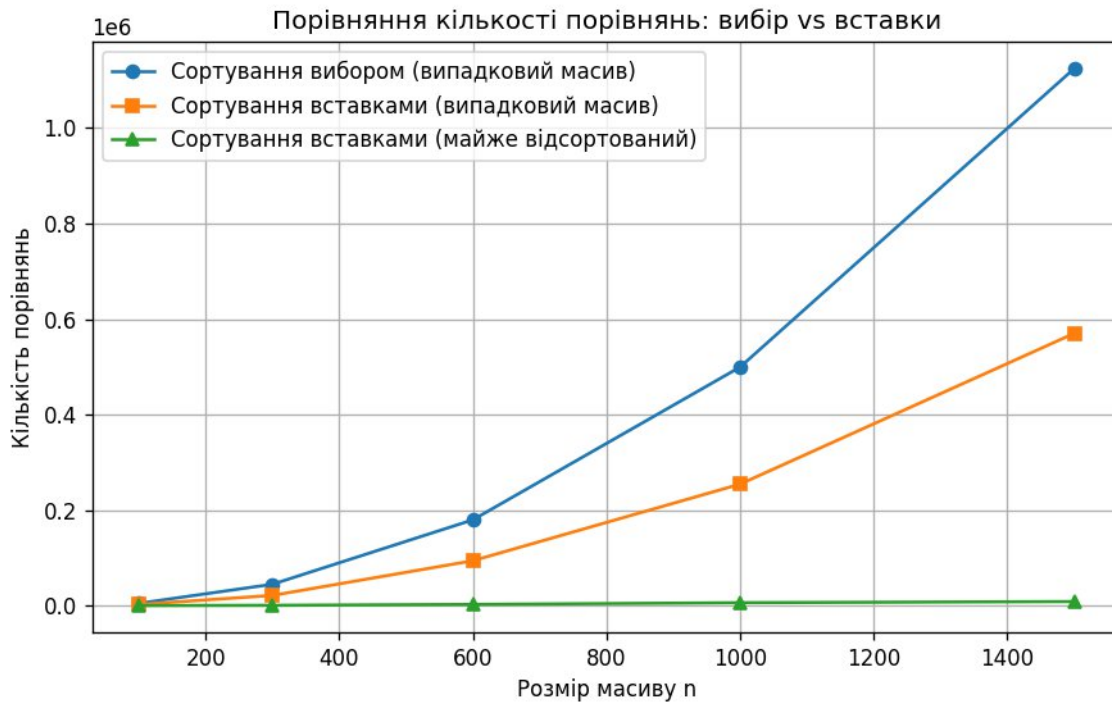
```

Результат виконання програми:

Сортування вибором: [1, 2, 3, 5, 5, 6, 9]
 Сортування вставками: [1, 2, 3, 5, 5, 6, 9]
 sorted() Python: [1, 2, 3, 5, 5, 6, 9]

n= 100: вибір(вип.)= 4950 вставки(вип.)= 2649 вставки(майже відсорт.)= 352
 n= 300: вибір(вип.)= 44850 вставки(вип.)= 21505 вставки(майже відсорт.)= 926
 n= 600: вибір(вип.)= 179700 вставки(вип.)= 94112 вставки(майже відсорт.)= 2775
 n= 1000: вибір(вип.)= 499500 вставки(вип.)= 254451 вставки(майже відсорт.)= 6304
 n= 1500: вибір(вип.)=1124250 вставки(вип.)= 569990 вставки(майже відсорт.)= 8688

Графік порівняння:



Висновок за результатами: на випадкових даних обидва алгоритми демонструють квадратичне зростання кількості порівнянь $\Theta(n^2)$, причому сортування вибором стабільно виконує приблизно вдвічі більше порівнянь, ніж сортування вставками (оскільки вибором завжди переглядає всю невідсортовану частину для пошуку мінімуму, тоді як вставки в середньому проходять лише половину вже відсортованої частини). Найпоказовіший результат — крива "вставки (майже відсортований масив)": кількість порівнянь зростає значно повільніше й лишається на порядки нижчою за інші криві, що є прямим підтвердженням адаптивності методу вставками: для майже впорядкованих даних його реальна складність наближається до $O(n)$.

Контрольні питання

1. За якими основними критеріями класифікують алгоритми сортування?
2. Сформулюйте теоретичну нижню межу складності порівняльних алгоритмів сортування. Чому вона дорівнює саме $\Omega(n \log n)$?
3. Що таке стійкість алгоритму сортування? Наведіть приклад стійкого й нестійкого алгоритму.
4. Що означає властивість "in-place" алгоритму сортування?
5. Що таке адаптивність алгоритму сортування? Який з вивчених у роботі алгоритмів є адаптивним?
6. Чому сортування вибором вважається нестійким? Наведіть приклад ситуації, коли порядок однакових ключів порушується.
7. Чим внутрішнє сортування відрізняється від зовнішнього?
8. За яких умов непорівняльні алгоритми сортування (наприклад, лічильникове) можуть досягти складності $O(n)$?

Зміст звіту

1. Тема, мета роботи.
2. Заповнена таблиця класифікації (завдання 1).
3. Ручний розрахунок і аналіз стійкості (завдання 2).

4. Тексти програм та результати їх роботи (завдання 3, 4).
5. Графік порівняння алгоритмів сортування.
6. Висновки.